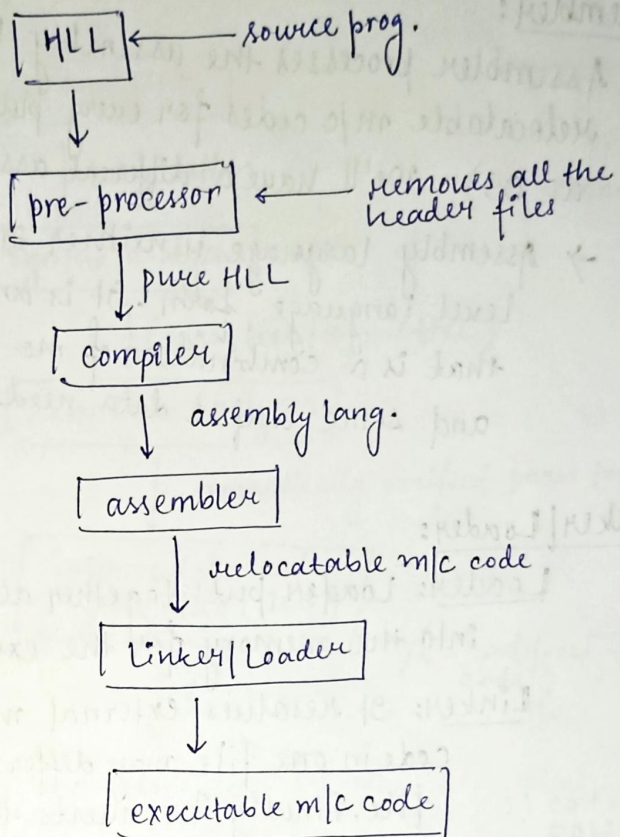


02/02/26

Language Processors

HLL - High level lang.

Phases of lang.
processing
systems



2 types of machine code:

- 1) relocatable m/c code
- 2) executable m/c code

High level language:

These are the human understandable languages written mostly in English or any other language. It contains header files such as `#include`, `#define`.

Pre-processor:

In this phase, all the header files are removed. `#include` directives are removed by including files called file inclusion and `#define` directives are removed using macro expansion.

Pre-processor generates pure high level lang. that is further fed to a compiler.

Compiler:

It is a language translator that reads the whole program at once. It is faster than the interpreter. It further generates assembly language.

Assembler:

Assembler processes the assembly languages and generates relocatable m/c codes for every platform (hardware as well as OS). We'll have a different assembler

→ Assembly language is neither in binary form nor in high level language form. It is an intermediate representation that is a combination of m/c instructions, registers and some useful data needed for executions.

Linker/Loader:

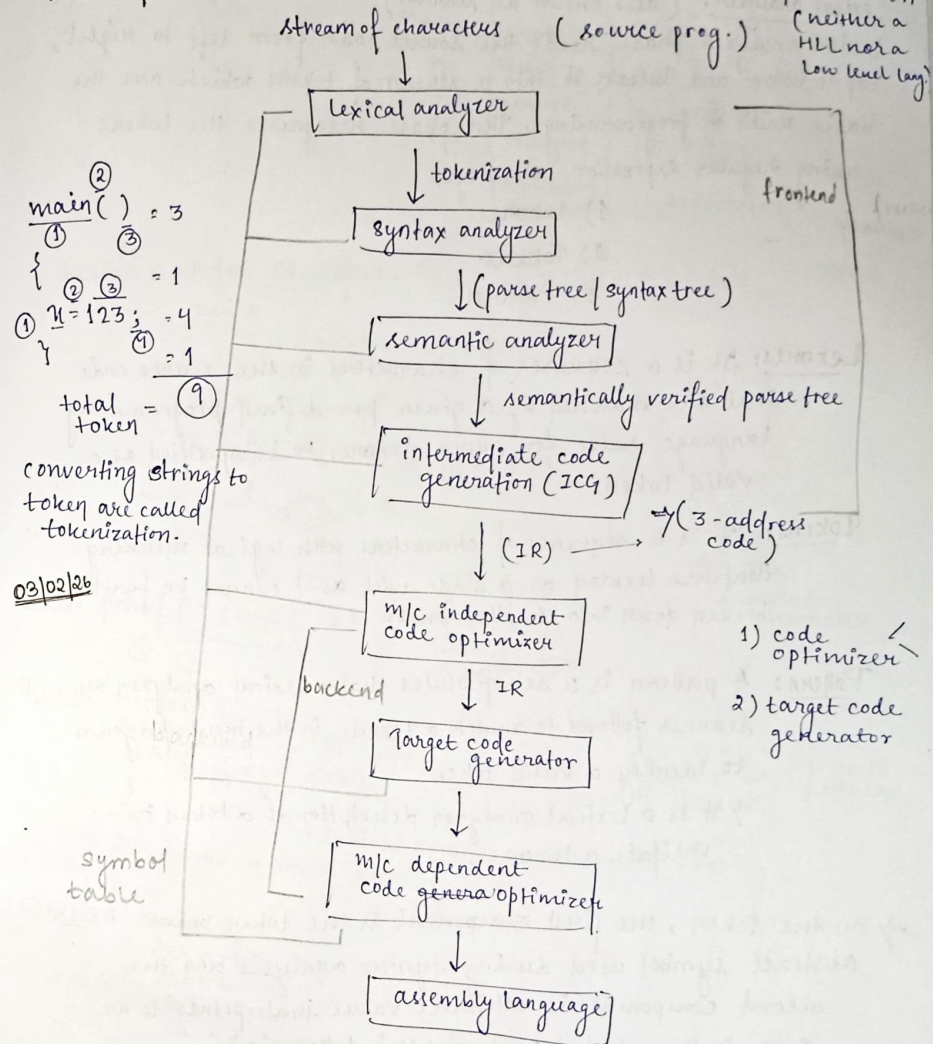
Loader: Loader puts together all the executable object files into the memory for the execution.

Linker: It resolves external memory addresses where a code in one file may differ to the location in another file. Finally it converts the relocatable m/c codes into executable m/c code or absolute m/c code.

Questions:

- 1) write down the difference betn interpreter and compiler with examples.
- 2) define file inclusion and macro expansion.
- 3) write down the difference betn relocatable m/c code and executable m/c codes.

Phases of compiler:



Lexical analyzer: (also known as scanner)

Lexical analyzer phase reads the source code from left to right, top to bottom and breaks it into a stream of tokens which are the basic units of programming. This phase recognises the tokens using regular expression or regex.

1) Lexemes

2) Tokens

3) Pattern

Lexemes: It is a sequence of characters in the source code that are matched by a given pre-defined programming language rules for every lexeme to be specified as a valid token.

Tokens: It is a sequence of characters with logical meaning that are treated as a single unit as it cannot be further broken down into smaller ones.

Pattern: A pattern is a set of rules that a lexical analyzer or scanner follows to match a lexeme in the input program to identify a valid token.

→ It is a lexical analyzer description of a token to validate a lexeme.

→ In the token, the first component is the token name i.e. an abstract symbol used during syntax analysis and the second component is attribute value that points to an entry in the symbol table for this token.

{token name, attribute value}

types of token:

- 1) Identifiers / keywords
- 2) Separators
- 3) Operators
- 4) Constants
- 5) Special characters

example:

1) `int n=10;`

lexeme	token	pattern
int	keyword	<code>i-n-t</code>
n	identifier	<code>[a-z]</code>
=	operator	<code>=, operator</code>
10	constant	<code>[0-9]</code>
;	separator/punctuator	<code>;, separator</code>

design a FA for identifier.

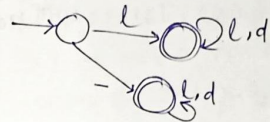
main

→ underscore

$l \rightarrow [a-z], [A-Z]$

$d \rightarrow [0-9]$

$RE = l(l+d)^* | _ (l+d)^*$



Ques1: `printf("i = %d, &i = %u", i, &i);`

$\&i$ → special character

Ques2: `main() { a = b + t + 1; -- ++ t + t = 1; printf("xyz, a, b); }`

`+=` ✓
`++` ✓
`--` ✓
} valid tokens

→ lexical error (double quote missing)

if double quote is there

then 25

Ques3: `main() { int a=10; char b="abc"; int c=30; char d="xyz"; int /* comment */ t m=40.5; }`

Ques 4:

```
main() {
    int a = 10;
    char b = "abc";
    int c = 30;
    char d = "xyx";
    int /*
```

05/02/26

Ques 5:

```
x == y / * abcd * * / * abcd */;
```


 ① ② ③ ④ x not counted ⑤ ⑥ ⑦ ⑧ ⑨
 total = 9

Ques 6:

```
int **p;
```


 ① ② ③ ④ ⑤ total = 5

Ques 7:

```
x = *p ++ ++ ++ y;
```


 ① ② ③ ④ ⑤ ⑥ ⑦ ⑧ ⑨ total = 9

Ques 8:

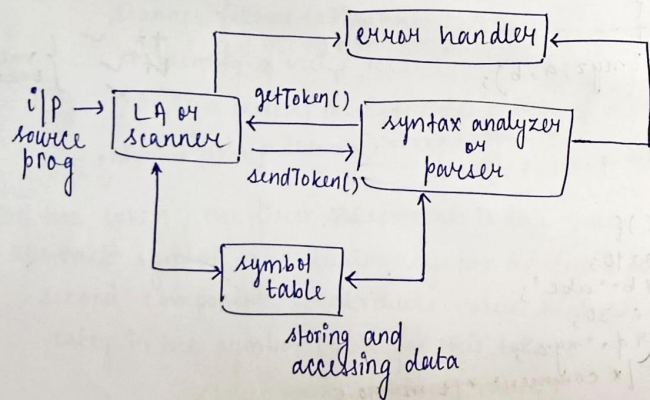
```
< = > = < = > = ! = < = >
```


 ① ② ③ ④ ⑤ ⑥ ⑦ ⑧ ⑨ total = 9

Note: Here & and * is counted as individual a single count

comment line will not be counted

Generation of tokens



input buffering with example

LA starts the process of tokenization once it receives the request from the parser.

The first phase that will get activated is syntax analyzer after getting the request, LA scans the code and converts it into tokens.

LA marks a symbol as a valid token only after looking at the next symbol.

It always finds the longest matching valid sequence and counted as a token.

$x \neq y$

- $x \rightarrow$ accept, can be a valid token
- $\neq \rightarrow$ not a valid token
- $x \rightarrow$ declare x as a valid token
- $\neq \rightarrow$ accept, can be a valid token
- $\neq \rightarrow$ accept, can be a valid token
- $\neq y \rightarrow$ not a valid token
- $\neq \rightarrow$ declare this as a valid token
- $y \rightarrow$ accept, can be a valid token
- $y \rightarrow$ declare y as a valid token

i/p buffering

Symbol Table: It is a data structure containing a record for each token having token name and attribute value. This table is created to store information about the occurrences of various entities such as variable and function names, objects, classes, interfaces.

Whenever LA encounters any identifiers, it creates an entry for that identifier in the symbol table.

Syntax analyzer stores the information such as scope, type, line of reference, line of usage.

19/02/24

Lexical error:

- Exceeding the length of identifier or numeric constants.
- Appearance of illegal characters. eg: \$
- Spelling error
- Unmatched string

Lexical error recovery:

- Imp
- Panic mode recovery: We delete successive characters from the remaining input until the lexical analyzer can find a well formed token at the beginning of what input is left.
eg: int ~~xyx~~; ^{remaining int;} → ek ke baad ek successive char ko delete krta jayega

• Inserting missing character:

eg: ~~it~~ n=10;
→ isko woh change krke int krdega.
(data type v change ho jayega)

• Delete any extra or unknown character:

eg: ~~intt~~ n=10;
→ isko change krega int ko
(delete krdega)

Ques:

G: $E \rightarrow E + E \mid id$

i/p: $id + id + id$

LMD1: $E \rightarrow E + E$

→ $id + E + E$

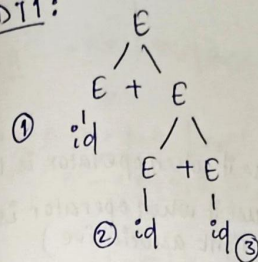
→ $id + id + id$

LMD2: $E \rightarrow E + E$

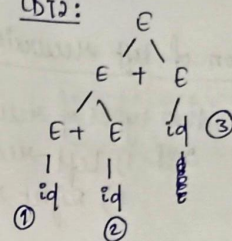
→ $E + E + E$

→ $id + id + id$

LDT1:



LDT2:



logically LDT2 is correct because:

jab hum $5+3+2$ kr rhe hain toh pehle $(5+3)$ evaluate horha hai then 2 add ho sha hai. i.e. $(5+3)+2=10$
(following left associativity rule for + operator).
jo ki LDT2 mein ho sha hai.

Ques: G: $E \rightarrow E + E \mid E * E \mid id$
i/p: $id * id + id$

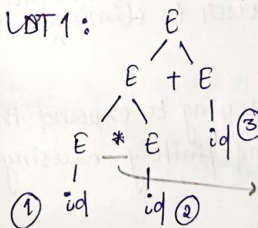
LMD1: $E \rightarrow E + E$

→ ~~id + id~~
→ $E * E + E$
→ $id * E + E$
→ $id * id + id$

LMD2: $E \rightarrow E * E$

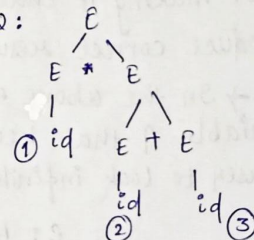
→ ~~id * id~~
→ $id * E + E$
→ $id * id + id$

LDT1:



→ as sabse niche hai so more precedence

LDT2:



Logically LDT1 is correct because precedence of multiplication is higher than addition.

LDT1: $(5 * 3) + 2$

LDT2: $5 * (3 + 2)$

→ Operator having higher precedence will be placed farther from the root node in the parse tree or it will be at lower level.

Elimination of left recursion:

There are two types of recursion:

- 1) Left recursion (use it when operator is left associative)
- 2) Right recursion (use it when operator is right associative)

Ques: $G: A \rightarrow A\alpha | B$

ye dono same hue toh isko left recursion bolte hain. $w = \{ B, B\alpha, B\alpha\alpha, B\alpha\alpha\alpha, \dots \}$
 $RE = B\alpha^*$

$A \rightarrow \alpha A | B$

ye dono same rahega toh ye right recursion hai

left recursion mein infinite loop ke chances jyada hain coz starting mein woh terminal ko dekh rha hai

$w = \{ B, \alpha B, \alpha\alpha B, \alpha\alpha\alpha B, \dots \}$
 $RE = \alpha^* B$

right recursion mein infinite loop ka chances kam hai because starting mein non-terminal hai.

Why eliminate left recursion from grammar?

→ left recursion can lead to infinite loop in certain parsing algo. making it challenging for the parser to ^{terminate} eliminate and produce correct results.

→ In the above example, we are trying to expand the variable A that keep calling itself indefinitely causing the parser to look infinite times.

$G: A \rightarrow A\alpha | B \leftarrow \text{left recursive } G$

↓

$G': A \rightarrow BA' \leftarrow \text{right recursive } G$

$A' \rightarrow \epsilon | \alpha A'$

second prod. rule dekh ke pata chal rha hai ke ye right recursive hai

Ques: $G: A \rightarrow A\alpha_1 | A\alpha_2 | A\alpha_3 \dots | B_1 | B_2 | B_3$

↓

$G': A \rightarrow B_1 A' | B_2 A' | B_3 A'$

$A' \rightarrow \epsilon | \alpha_1 A' | \alpha_2 A' | \alpha_3 A'$

left recursion remove krke right recursion bana rha hai

Ques: $G: E \rightarrow E+T | T$
 $T \rightarrow a$

$G': E \rightarrow TE'$
 $E' \rightarrow \epsilon | +TE''$
 $T \rightarrow a$

isme jo left recursion hai usko remove krke right recursion bana rha hai

Ques: $G: L \rightarrow L, S | S$

$G': L \rightarrow SL'$
 $L' \rightarrow \epsilon | , SL'$

Ques: $G: S \rightarrow S01S0S | 01$ $\Rightarrow G: S \rightarrow 01S'$
 $S' \rightarrow \epsilon | 01S0SS'$

$G: S \rightarrow 01S0S | 01$

alag hai toh isme left recursion hai hi nhi

Ques: $G: E \rightarrow E+T | T$
 $T \rightarrow T * F | F$
 $F \rightarrow a$

$G': E \rightarrow TE'$
 $E' \rightarrow \epsilon | +TE'$
 $T \rightarrow FT'$
 $T' \rightarrow \epsilon | * FT'$
 $F \rightarrow a$

Ans

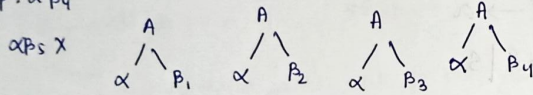
Ques: $G: E \rightarrow E * n | E + n | n$

Converting non-deterministic grammar to deterministic grammar using left factoring:-

- If we find out common prefix in the production rules then the grammar is non-deterministic grammar.
- The common prefix should be present for the same variable not across different variables.

$$G: A \rightarrow \alpha B_1 | \alpha B_2 | \alpha B_3 | \alpha B_4 | \dots$$

$$i|P: \alpha B_4$$



Because of the common prefix α , we have to do a hit and trial process to reach to the production rule αB_4 . We have multiple choices to reach to that prod. rule. This is non-deterministic grammar.

agar $G: A \rightarrow \alpha B_1$ hota toh α jo ki common hai usse koi farak nhi padta.

$$G: A \rightarrow \alpha B_1 | \alpha B_2 | \alpha B_3 | \alpha B_4$$

↓

$$G': A \rightarrow \alpha A' \\ A' \rightarrow B_1 | B_2 | B_3 | B_4$$

Ques: $G: S \rightarrow iEtS | iEtSeS | a$
 $E \rightarrow b$

↓

$$G': S \rightarrow iEtSS' | a \\ S' \rightarrow e | eS \\ E \rightarrow b$$

Given this grammar, identify non-deterministic grammar and if exist convert it to deterministic grammar.

Ques: $G: S \rightarrow aSSb | aSaSb | abb | b$

↓

agar aS common lenge toh 2 prod. rule cover hoga but if 'a' ko common lenge toh 3 prod. rule cover ho jayega.

$$S' \rightarrow SSbS | SaSb | bb$$

$$S' \rightarrow SS'' | bb$$

$$S'' \rightarrow SbS | aSb$$

'S' ko common liya hai

$$G'': S \rightarrow aS' | b$$

$$S' \rightarrow SS'' | bb$$

$$S'' \rightarrow SbS | aSb$$

'aS' ko common liya hai

← $\alpha = aS$

$$G': S \rightarrow aSS' | abb | b \\ S' \rightarrow SbS | aSb$$

$$\alpha = a \quad S \rightarrow aS'' | b \\ S'' \rightarrow SS' | bb$$

from the original grammar hai → common prefix aa rha hai

Ques: $G: S \rightarrow a | ab | abc | abcd$

$$G': S \rightarrow aS'$$

$$S' \rightarrow e | b | bc | bcd$$

$$S' \rightarrow e | bS''$$

$$S'' \rightarrow e | c | cd$$

$$S'' \rightarrow e | cS'''$$

$$S''' \rightarrow e | d$$

$$\Rightarrow G'': S \rightarrow aS' \\ S' \rightarrow e | bS'' \\ S'' \rightarrow e | cS''' \\ S''' \rightarrow e | d$$

HW $G: G \rightarrow E * n | E + n | n$

24/02/26

Parser

(LMD ✓
left recursion x
NDG x)

Top down parser

Bottom up parser
SR parser (RMD in reverse order)

Recursive descent parser
(with back tracking)

* Predictive parser (LL(1))
(without back tracking)

operator precedence parser
(ambiguous)

* LR parser
LR(0) SLR(1) CLR(1) LALR(1)

LL(1) parser → look ahead symbol
→ used LMD
→ scan left to right

→ For constructing a parsing table for LL(1) parser, we need two functions: 1) first() 2) follow()

first(): first of any variable contains set of terminals in first position in every string derived by that variable.

→ first of any variable may contain 'ε'.

follow(): follow of any variable contains a set of all terminals present in the first position of every string placed immediately right after that variable.

→ follow of the starting symbol will always contain \$ symbol.

→ follow of any variable doesn't have any 'ε' in it.

eg: If $A \rightarrow \alpha B \beta$, Follow(B) = First(β)

first(β) does not contain ε.

If $A \rightarrow \alpha B$, Follow(B) = Follow(A)

Ques: $F_i()$ first()

$G: S \rightarrow aABC$	{a}
$A \rightarrow b$	{b}
$B \rightarrow c$	{c}
$C \rightarrow d$	{d}

$F_o()$ follow()

	{ \$ }
	{ c }
	{ d }
	{ \$ }

first() of any terminal is terminal itself

$F_o(B) = F_i(C)$

$= \{d\}$

$S \rightarrow aABC$
 $\rightarrow aABd$

$F_o(A) = F_i(B)$

$= \{c\}$

$S \rightarrow aABC$
 $\rightarrow aAcC$

toh isilge $F_o(A) = c$ hua

S
 $\swarrow$ $\downarrow$ $\searrow$
 a A B C
 $F_i(S) = F_i(aABC)$
 $= F_i(a)$
 $= \{a\}$ ✓

Ques:

$G: S \rightarrow aBDh$
 $B \rightarrow cC$
 $C \rightarrow bc | e$
 $D \rightarrow EF$
 $E \rightarrow gl | e$
 $F \rightarrow fle$

1) $D \rightarrow EF$
 $F_i(D) \rightarrow F_i(E)$
 $\rightarrow \{g, \epsilon\}$

2) $D \rightarrow EF$
 $\rightarrow \epsilon F$
 $\rightarrow F$
 $F_i(D) \rightarrow F_i(F)$
 $\rightarrow \{f, \epsilon\}$

3) $D \rightarrow \epsilon$

$F_i()$

{a}
{c}
{b, ε}
{g, f, ε}
{g, ε}
{f, ε}

$F_o(B) = F_i(B)$

$= \{g, f, \epsilon\}$

1) $S \rightarrow aBgh$

2) $S \rightarrow aBfh$

3) $S \rightarrow aBeh$
 $\rightarrow aBh$

$F_o()$

{ \$ }
{g, f, h}
{g, f, h}
{h}
{f, h}
{h}

$F_o(E) = F_i(F)$

$= \{f, \epsilon\}$

1) $D \rightarrow EF$

$\rightarrow \epsilon F$

2) $D \rightarrow \epsilon E$

$\rightarrow \epsilon E$

$F_o(E) = F_o(D)$

Ques:

$G: E \rightarrow E+T | T$
 $T \rightarrow T * F | F$
 $F \rightarrow id | (E)$

left recursive
ko, phle hatana
hai then $F_i()$ and $F_o()$
nikalna hai warna
answer nhi aayega

→ $G': E \rightarrow TE'$
 $E' \rightarrow \epsilon | +TE'$
 $T \rightarrow FT'$
 $T' \rightarrow \epsilon | *FT'$
 $F \rightarrow id | (E)$

Terminals: {+, *, id, (,)}

26/03/26

Q: $E \rightarrow TE'$
 $E' \rightarrow \epsilon | +TE'$
 $T \rightarrow FT'$
 $T' \rightarrow \epsilon | *FT'$
 $F \rightarrow id | (E)$

$fi()$
 $\{id, (\}$
 $\{ \epsilon, + \}$
 $\{id, c \}$
 $\{ \epsilon, * \}$
 $\{id, c \}$

$fo()$
 $\{ \epsilon,) \}$
 $\{ \epsilon,) \}$
 $\{ +, \epsilon,) \}$
 $\{ +, \epsilon,) \}$
 $\{ *, +, \epsilon,) \}$

$fi(E) = fi(T)$
 $= fi(F)$
 $= id$

$fo(E) = fi(E)$

$fo(E') = fo(E)$
 $= \{ \epsilon,) \}$

$fo(E') = fo(E')$
 no meaning of this so
 ye wapas jayega
 and $fo(E') = fo(E)$
 hoga.

$fo(T) = fi(E')$
 $= \{ \epsilon, + \}$ ①

$E \rightarrow TE'$
 $\rightarrow T+$

$E \rightarrow TE'$
 $\rightarrow TE$

merge ① and ②, then $fo(T)$ will get.
 $fo(T) = fo(E)$
 $= \{ \epsilon,) \}$ ②

$fo(F) = fi(T')$
 $= \{ \epsilon, * \}$ ③

$T \rightarrow FT'$
 $\rightarrow F*$

$fo(F) = fo(T)$
 $= \{ +, \epsilon,) \}$ ④

merge ③ and ④, will get $fo(F)$.

$fo(T') = fo(T)$
 $= \{ +, \epsilon,) \}$

Ques: $G: S \rightarrow AaAb | BbBa$
 $A \rightarrow \epsilon$
 $B \rightarrow \epsilon$

$fi()$
 $\{a, b \}$
 $\{ \epsilon \}$
 $\{ \epsilon \}$

$fo()$
 $\{ \epsilon \}$
 $\{a, b \}$
 $\{b, a \}$

$fi(S) = fi(AaAb)$
 $= fi(\epsilon a A b)$
 $= fi(a A b)$
 $= a$

$fi(S) = fi(BbBa)$
 $= fi(\epsilon b B a)$
 $= fi(b B a)$
 $= b$

$fo(S) = \epsilon$
 $fo(A) = fi(AaAb)$
 $= \{a, b \}$

$fo(B) = fi(BbBa)$
 $= \{b, a \}$

Ques: $G: S \rightarrow Bb | Cd$
 $B \rightarrow aB | \epsilon$
 $C \rightarrow cC | \epsilon$

$fi()$
 $\{a, b, c, d \}$
 $\{a, \epsilon \}$
 $\{c, \epsilon \}$

$fo()$
 $\{ \epsilon \}$
 $\{ b \}$
 $\{ d \}$

$fi(S) = fi(Bb)$
 $= a$

$fi(S) = fi(Cd)$
 $= c$

$fi(S) = fi(Bb)$
 $fi(S) = fi(Cd) \rightarrow$ replace ϵ

Ques: $G: S \rightarrow ABCDE$
 $A \rightarrow a | \epsilon$
 $B \rightarrow \epsilon | b$
 $C \rightarrow c$
 $D \rightarrow d | \epsilon$
 $E \rightarrow \epsilon | e$

$fi()$
 $\{a, b, c \}$
 $\{a, \epsilon \}$
 $\{b, \epsilon \}$
 $\{c \}$
 $\{d, \epsilon \}$
 $\{e, \epsilon \}$

$fo()$
 $\{ \epsilon \}$
 $\{b, c \}$
 $\{c \}$
 $\{d, \epsilon, e \}$
 $\{e, \epsilon \}$
 $\{ \epsilon \}$

$fi(S) = ABCDE$

$fi(S) = fi(A)$
 $= \{a, \epsilon \}$

$S \rightarrow ABCDE$
 $\rightarrow \{a \}$

$S \rightarrow EBCDE$
 $\rightarrow \{b, \epsilon \}$

$S \rightarrow \epsilon ECDE$
 $\rightarrow \{c \}$

ab yahan ruk
 jayega coz aur ϵ hai

Ques: $G: S \rightarrow ASA | AB | bA$
 $A \rightarrow aB | dS$
 $B \rightarrow b | \epsilon$

$fi()$
 $\{a, b, d \}$
 $\{a, d \}$
 $\{b, \epsilon \}$

$fo()$
 $\{ \epsilon, a, b, d \}$
 $\{ \epsilon, a, b, d \}$
 $\{ \epsilon, a, b, d \}$

$fo(S) = fi(A)$
 $= \{a, d \}$

$fo(S) = fo(A)$
 $= \{a, b, d \}$

$fo(A) = fi(S)$
 $= \{a, b, d \}$

$fo(A) = fo(S)$
 $= \{ \epsilon, a, b, d \}$

02/08/26

LL(1) Parsing table

Ques: $G: E \rightarrow TE'$
 $E' \rightarrow \epsilon | +TE'$
 $T \rightarrow FT'$
 $T' \rightarrow \epsilon | *FT'$
 $F \rightarrow id | (E)$

Given this grammar, if it is LL(1)
 parser or not.

$E \rightarrow TE'$
 $E' \rightarrow \epsilon | +TE'$
 $T \rightarrow FT'$
 $T' \rightarrow \epsilon | *FT'$
 $F \rightarrow id | (E)$

$fi()$
 $\{id, c \}$
 $\{ \epsilon, + \}$
 $\{id, c \}$
 $\{ \epsilon, * \}$
 $\{id, \epsilon \}$

$fo()$
 $\{ \epsilon,) \}$
 $\{ \epsilon,) \}$
 $\{ +, \epsilon,) \}$
 $\{ +, \epsilon,) \}$
 $\{ \epsilon, * \}$

	id	+	*	(	)	\$
E	$E \rightarrow TE'$			$E \rightarrow TE'$		
E'		$E' \rightarrow +TE'$			$E' \rightarrow \epsilon$	$E' \rightarrow \epsilon$
T	$T \rightarrow FT'$			$T \rightarrow FT'$		
T'		$T' \rightarrow \epsilon$	$T' \rightarrow *FT'$		$T' \rightarrow \epsilon$	$T' \rightarrow \epsilon$
F	$F \rightarrow id$			$F \rightarrow ($		

pehle $E \rightarrow TE'$ ka dekhenge use TE' mein left side yani T ko lenge. T ka $fi()$ ko check krenge. T ka $fi() \rightarrow \{id, (\}$ hai toh (E, id) aur $(E, ($ jahan par hoga wahan par prod. rule likhenge. $T' \rightarrow \epsilon$ waliye $fo()$ ko check krenge. as T' ka $fo \rightarrow \{+, *, \}$ hai toh $(T', +)$, $(T', *)$ aur (T', ϵ) jahan par hoga wahan par prod. rule likh denge.

right hand side ka first find out karna hai

$$\textcircled{1} E \rightarrow TE', fi(T) = \{id, (\}$$

$(E, id), (E, ($
row col row col

$$T' \rightarrow *FT'$$

ϵ hai toh follow find krenge

$$\textcircled{2} E' \rightarrow +TE'$$

$$\textcircled{3} T \rightarrow FT' \quad fi(F) = \{id, (\}$$

there is no multiple entry in any of the cells of the LL(1) parsing table.

So, we can say that the grammar is LL(1) parser.

Ques:

$G: S \rightarrow aBDh$	$fi()$	$fo()$
$B \rightarrow cC$	$\{a\}$	$\{\phi\}$
$C \rightarrow bC \epsilon$	$\{c\}$	$\{g, f, h\}$
$D \rightarrow EF$	$\{b, \epsilon\}$	$\{g, f, h\}$
$E \rightarrow g \epsilon$	$\{g, f, \epsilon\}$	$\{h\}$
$F \rightarrow f \epsilon$	$\{g, \epsilon\}$	$\{f, h\}$
	$\{f, \epsilon\}$	$\{h\}$

$D \rightarrow EF$ mein E left mein hai toh usko dekhenge usme ab check krenge E ka $fi()$. E ka $fi() \rightarrow \{g, \epsilon\}$ so, (E, g) ab E aur ϵ ka dekhenge, toh ϵ ke liye $fo()$ check krenge E ka $fo() \rightarrow \{f, h\}$ so, (E, f) and (E, h) ko lenge.

	a	b	c	g	f	h	\$
S	$S \rightarrow aBDh$						
B			$B \rightarrow cC$				
C		$C \rightarrow bC$		$C \rightarrow \epsilon$	$C \rightarrow \epsilon$	$C \rightarrow \epsilon$	
D				$D \rightarrow EF$	$D \rightarrow EF$	$D \rightarrow EF$	
E				$E \rightarrow g$	$E \rightarrow \epsilon$	$E \rightarrow \epsilon$	
F					$F \rightarrow f$	$F \rightarrow \epsilon$	

$$fi(D) = fi(E) = \{g, \epsilon\}$$

$$D \rightarrow EF$$

$$g \leftarrow$$

$$D \rightarrow EF$$

$$\epsilon \leftarrow$$

There is no multiple entry in grammar so it is LL(1) parser.

Stack operation for LL(1) parser:

$$G: E \rightarrow TE'$$

$$E' \rightarrow \epsilon | +TE'$$

$$T \rightarrow FT'$$

$$T' \rightarrow \epsilon | *FT'$$

$$F \rightarrow id | ($$

i/p - $id + id \$$ (given this, find out if it is generating this i/p or not)

Stack

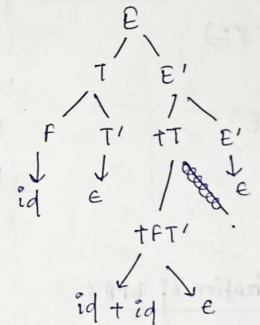
$\$E$
 $\$E'T$
 $\$E'T'F$
 $\$E'T' \uparrow$
 $\$E'T'$
 $\$E'(E)$
 $\$E'$
 $\$E'T \times$
 $\$E'T$
 $\$E'T'F$
 $\$E'T' \uparrow$
 $\$E'T'$
 $\$E'(E)$
 $\$E'$
 $\$($

i/p

$id + id \$$
 $id + id \$$
 $id + id \$$
 $id + id \$$
 $+id \$$
 $+id \$$
 $+id \$$
 $\times id \$$
 $id \$$
 $id \$$
 ϕ
 ϕ
 ϕ
 ϕ

action

$E \rightarrow TE'$ (push in reverse order)
 $T \rightarrow FT'$ (push)
 $F \rightarrow id$ (push)
pop
 $T \rightarrow \epsilon$
—
 $E' \rightarrow +TE'$
pop
 $T \rightarrow FT'$
 $F \rightarrow id$
pop
 $T' \rightarrow \epsilon$
—
 $E' \rightarrow \epsilon$
accept



05/03/26

Ques: $G: S \rightarrow aBDh$
 $B \rightarrow cC$
 $C \rightarrow bC | \epsilon$
 $D \rightarrow \epsilon F$
 $E \rightarrow g | \epsilon$
 $F \rightarrow h | \epsilon$

i/p: ~~acbh~~

Stack

$\$S$
 $\$hDB\$$
 $\$hDB$
 $\$hDC$
 $\$hDC$
 $\$hDC$
 $\$hDC$
 $\$hDC$
 $\$hD(E)$
 $\$hD$
 $\$hF(E)$
 $\$hF$
 $\$h(E)$
 $\$h$
 $\$$

i/p

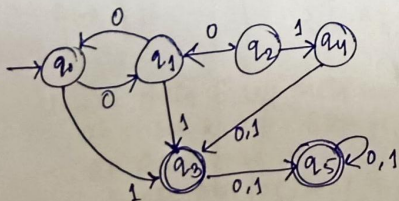
$acbh\$$
 $cbh\$$
 $cbh\$$
 $cbh\$$
 $bh\$$
 $bh\$$
 $bh\$$
 $h\$$
 $h\$$
 $h\$$
 $h\$$
 $h\$$
 $h\$$
 $h\$$
 $h\$$
 $h\$$
 $\$$

action

$S \rightarrow aBDh$
 pop
 $B \rightarrow cC$
 pop
 $C \rightarrow bc$
 pop
 $C \rightarrow \epsilon$
 -
 $D \rightarrow \epsilon F$
 $E \rightarrow \epsilon$
 -
 $F \rightarrow \epsilon$
 -
 pop
 accept

Minimization of DFA:

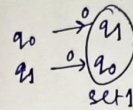
Ques: Given a DFA, minimize it.



unreachable states: $\{q2, q4\}$
 remaining states: $\{q0, q1, q3, q5\}$

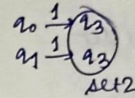
[set of non-final states]
set1

$\pi_0 = [q0, q1]$
 (0-equivalence)



[set of final states]
set2

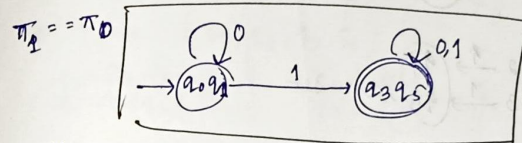
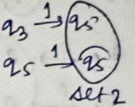
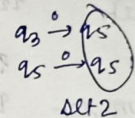
$[q3, q5]$



$q0$ and $q1$ showing same behaviour meaning belongs to same set separately for individually so we will keep them together in a single group in the next step.

$\pi_1 = [q0, q1]$

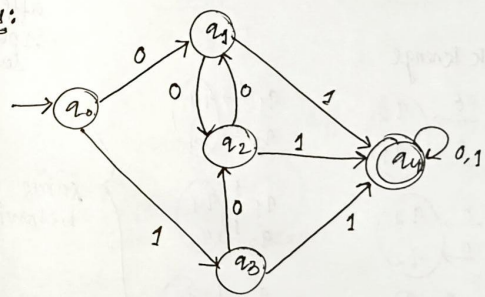
$[q3, q5]$



this is the minimized DFA

Steps: construct transition table
 eliminate unreachable states
 group the states into 2 i.e. final and non-final
 * if the no. of group ya states niklenge utne no. of states banenge minimized DFA mein.

Ques:



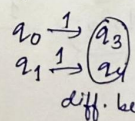
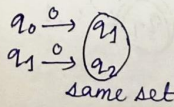
	0	1
$\rightarrow q0$	$q1$	$q3$
$q1$	$q2$	$q4$
$q2$	$q1$	$q4$
$q3$	$q2$	$q4$
* $q4$	$q4$	$q4$

set1 (non-final)

set2 (final)

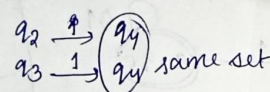
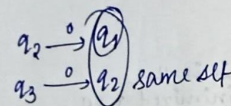
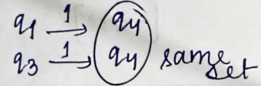
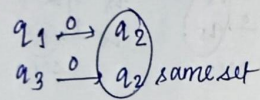
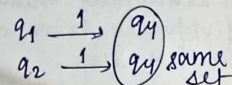
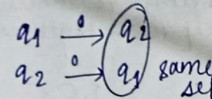
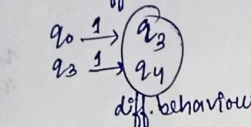
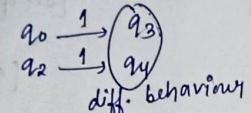
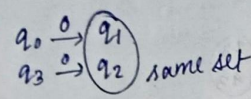
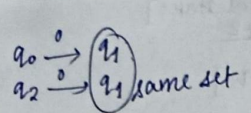
$\pi_0 = [q0, q1, q2, q3]$

$[q4]$



same set

diff. behaviour



Set 1

Set 2

Set 3

* $\pi_1 = [q_0]$

$[q_1, q_2, q_3]$

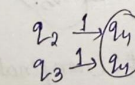
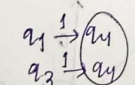
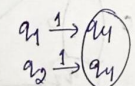
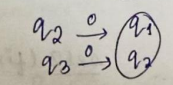
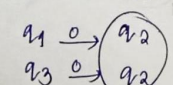
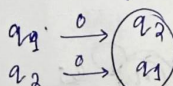
$[q_4]$

$\rightarrow \pi_2$

after further splitting yahi same hai

$[\pi_0 \neq \pi_1]$

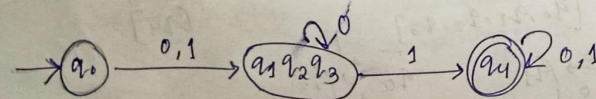
so further splitting check krenge
 previous π_0 se equal nhi hai toh tab tak split krenge jab tak previous π se match na ho jaye.



same behaviour

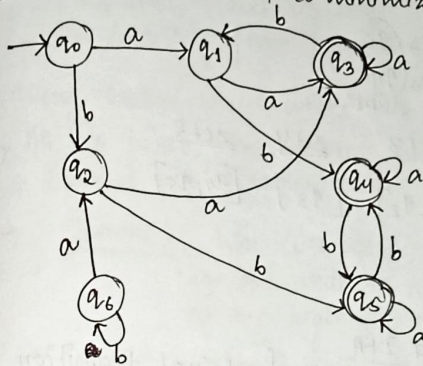
$\pi_2 = \pi_1$

final minimized DFA



10/03/26

Ques: Convert this DFA to minimize DFA.



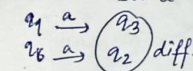
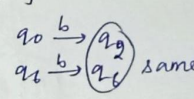
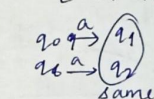
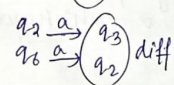
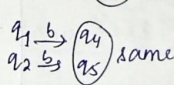
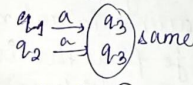
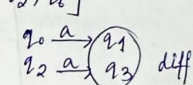
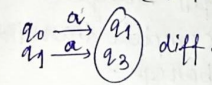
	a	b
$\rightarrow q_0$	q_1	q_2
q_1	q_3	q_4
q_2	q_3	q_5
* q_3	q_3	q_4
* q_4	q_4	q_5
* q_5	q_5	q_4
q_6	q_2	q_6

Set 1 (non-final)

Set 2 (final)

* $\pi_0 = [q_0, q_1, q_2, q_6]$

$[q_3, q_4, q_5]$



Set 1

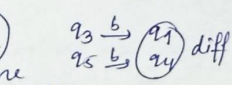
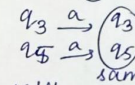
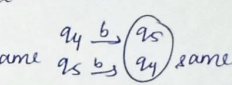
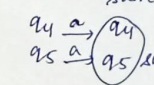
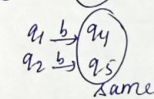
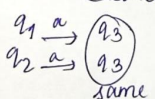
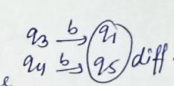
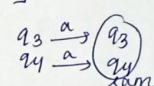
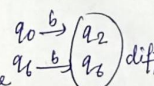
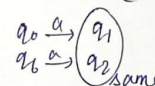
Set 2

Set 3

* $\pi_1 = [q_0, q_6]$

$[q_1, q_2]$

$[q_3, q_4, q_5]$



Set 1

Set 2

Set 3

Set 4

Set 5

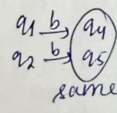
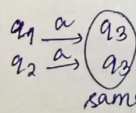
* $\pi_2 = [q_0]$

$[q_6]$

$[q_1, q_2]$

$[q_3]$

$[q_4, q_5]$



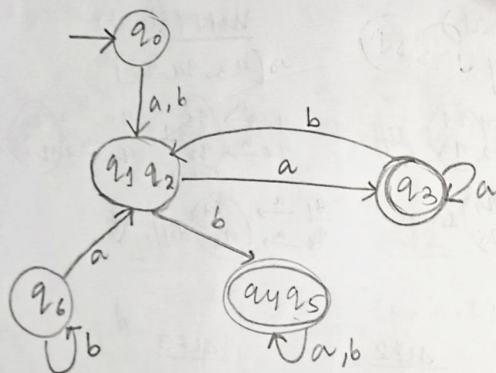
$q_4 \xrightarrow{a} q_4$
 $q_5 \xrightarrow{a} q_5$
 same

$q_4 \xrightarrow{b} q_5$
 $q_5 \xrightarrow{b} q_4$
 same

$* \pi_3 = \begin{matrix} \text{set1} & \text{set2} & \text{set3} & \text{set4} & \text{set5} \\ [q_0] & [q_6] & [q_1, q_2] & [q_3] & [q_4, q_5] \end{matrix}$

$\pi_2 = \pi_3$

final minimized DFA



original transition
 table dekh ke isko
 fill krnge
 complete transition
 banega.
 aur original mein
 jo final the isme
 v wahi hoga.

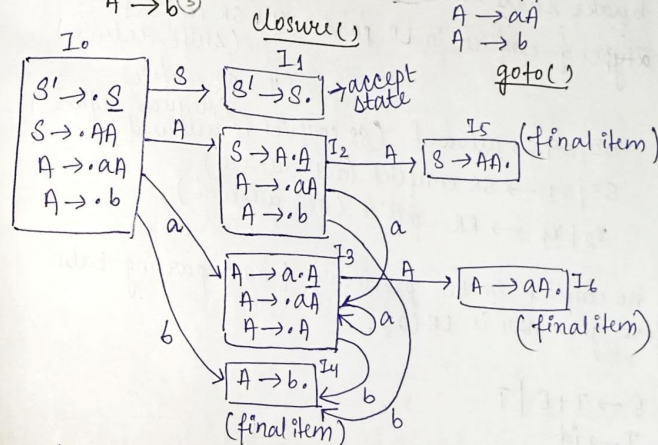
imp (8 marks)
 LR(0) parser → look ahead symbol
 ↳ RMD in reverse order
 ↳ Scanning from left to right

→ Every LR parser works with unambiguous grammar.
 → All LR parser contain same parsing algorithm but they will have different parsing tables.

→ closure(): whenever we find closure() of any LR(0) item or symbol whatever new prod. is added just put a dot at the beginning of the LR(0) item.
 → Goto(): we will right shift all the dots to the extreme right end until they are completely reduced.

Quest 1: $G: S \rightarrow AA$ → augmented G
 $A \rightarrow aa$
 $A \rightarrow b$

$G: S' \rightarrow S$
 $S \rightarrow AA$
 $A \rightarrow aa$
 $A \rightarrow b$
goto()



pehle left side pe dot lagayenge then if terminal hai toh wahan pe ruk jayenge but agar non-terminal ya variable hoga toh further goto() nikalenge.
 pehle I0 ko complete krenge ek ek prod. rule ka nikalke closure() then uske baad I1 jayenge fir ek ek krke saare complete krenge. Last mein dot aagaye toh woh final item ho jayega. agar koi repeat ho rha hai toh usko firse na ikhake arrow laga denge.

12/03/21

LR(0) parsing table

state no.	action			GoTo	
	a	b	\$	S	A
0	S3	S4		1	2
1			accept		
2	S3	S4			5
3	S3	S4			6
4	r3	r3	r3		
5	r1	r1	r1		
6	r2	r2	r2		

S → shift
r → reduction

pehle har state ka transition check krenge aur usko likhenge
I₀ → I₃ state ko ja rha hai so table mein S3 likhenge S denote krta hai shift ko. then GoTo check krenge I₀ ka like I₀ → I₁ pe ja rha hai toh GoTo mein sirf state no. likhenge.
I₁ khud ek final state hai aur woh aage nhi badh rha toh \$ mein accept likh denge.
aise hi baaki kh krenge.

I₄, I₅, I₆ tino final item hai toh reduction ho rha hai.

ab I₄ mein prod. rule 3 hai toh action mein saare (\$ tak) sabmein r₃ likhenge r₃ mtlb reduction on 3rd prod. rule.

S → AA ①
A → aA ②
A → b ③
aise hi baaki saare ka

likhenge

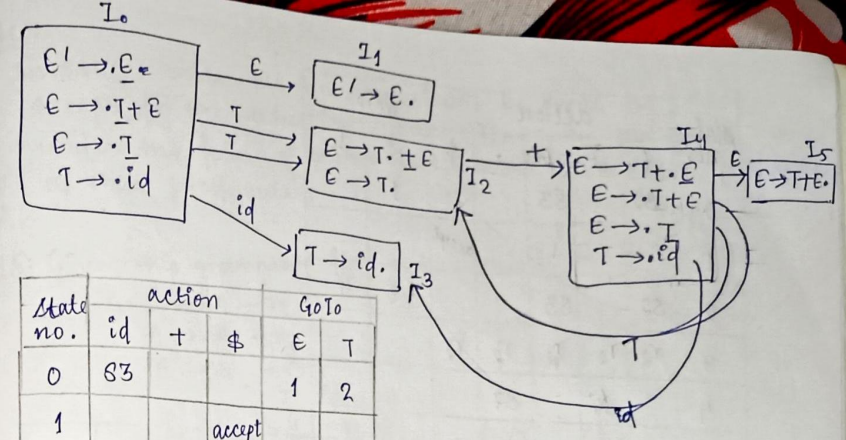
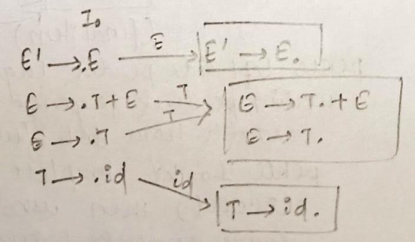
→ there are 2 types of conflict in LR parser (1) SR conflict (shift reduce)

(II) RR conflict

- S3/S4 → allowed (SR conflict is allowed)
- S3/r1 → SR conflict (not allowed)
- r2/r1 → RR conflict (not allowed)

→ There is no conflict in the previously drawn parsing table therefore grammar is LR(0).

Ques 2: G: E → T + E | T
T → id
↓
augmented
G': E' → E
E → T + E
E → T
T → id

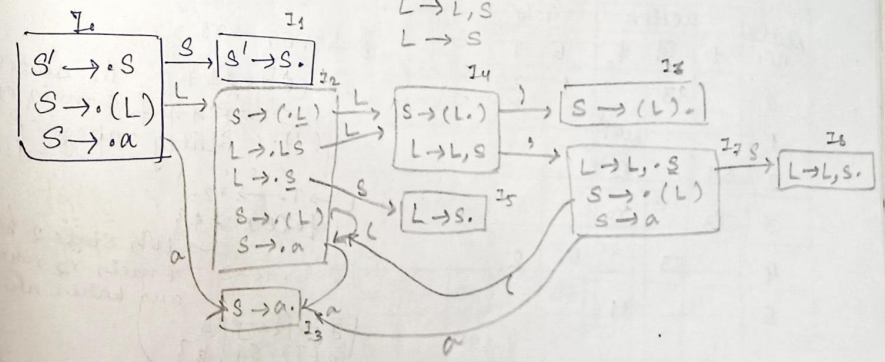


There is a multiple entry or SR conflict in the table in state I₂, so the grammar is not LR(0).

→ when there is a GoTo on same symbol, we will merge those states into a single state.

16/03/26

Ques 3: G: S → (L) / a ⇒ G': S' → S
S → (L)
S → a
L → L, S
L → S



state no.	action					GoTo	
	(	)	a	,	\$	S	L
0	S2		S3			1	
1					accept		
2	S2		S3			5	4
3	r2	r2	r2	r2	r2		
4		S6		S7			
5	r4	r4	r4	r4	r4		
6	r1	r1	r1	r1	r1		
7	S2		S3			8	
8	r3	r3	r3	r3	r3		

SLR(1) (Simple LR(1))

Quest 4: Given this grammar, find out if this is SLR(1) or not.

$G: E \rightarrow T + E \mid T$

$T \rightarrow id$

refer the diag. from previous quest.

The table does not contain multiple entries so it is SLR(1).

state no.	action				GoTo	
	+	id	\$		E	T
0	S3				1	2
1			accept			
2	S4		r2			
3	r3		r3			
4		S3			5	2
5			r1			

$E \rightarrow T + E. \rightarrow r1$

$fo(E) = \{ \$ \}$ in states
 $\hookrightarrow$ toh sirf \$ mein r1 entry rahega.

$E \rightarrow T. \rightarrow r2$

$fo(E) = \{ \$ \}$
 $\hookrightarrow$ toh state 2 ke \$ mein r2 rahega aur kahi nhi

$T \rightarrow id. \rightarrow r3$

$fo(T) = \{ +, \$ \}$
 $\hookrightarrow$ toh state 3 ke + and \$ mein r3 rahega

Note:

- In SLR(1) table, all the entries will be same as LR(0) except for the reduction moves. Place the reduction moves under the follow of variable present in the left hand side of that prod. rule.

17/03/24

Quest 5: Given this grammar find out if this is SLR(1) or not.

$G: S \rightarrow AA - r1$

$A \rightarrow aA - r2$

$A \rightarrow b - r3$

refer the diagram from quest 1.

state no.	action			GoTo	
	a	b	\$	S	A
0	S3	S4		1	2
1			accept		
2	S3	S4			5
3	S3	S4			6
4	r3	r3	r3		
5			r1		
6	r2	r2	r2		

$fo(S) = \{ \$ \}$

$fo(A) = \{ a, b, \$ \}$

$\hookrightarrow fo(A) = fi(A)$
 $= \{ a, b \}$

then again
 $fo(A) = fo(S)$
 $= \{ \$ \}$

$fo(A) = \{ a, b, \$ \}$

Quest 6: $G: S \rightarrow (L) \mid a$

$L \rightarrow L, S \mid S$

$G: S' \rightarrow S$

$S \rightarrow (L) - r1$

$S \rightarrow a - r2$

$L \rightarrow L, S - r3$

$L \rightarrow S - r4$

$fo(S) = \{ \$, (,) \}$

$fo(L) = \{), , \}$

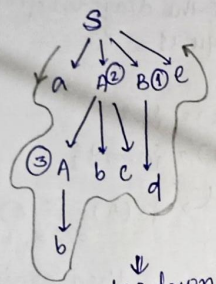
refer diagram from quest 3.

state no.	action					GoTo	
	(	)	a	,	\$	S	L
0	S2		S3			1	
1					accept		
2	S2		S3			5	4
3		r2		r2	r2		
4		S6		S7			
5		r4		r4			
6		r1		r1	r1		
7	S2		S3			8	
8		r3		r3			

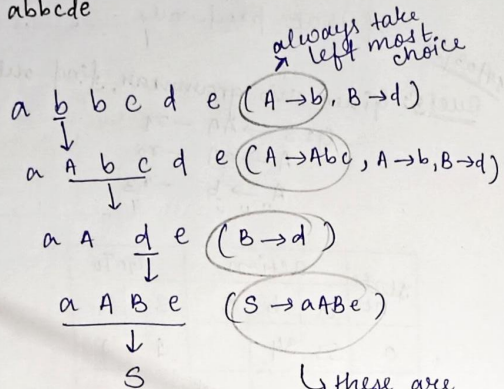
Handle Pruning

Ques: Given this grammar, derive the input string: abcde using RMD.
 i/p: abcde

G: $S \rightarrow ABe$
 $A \rightarrow Abc | b$
 $B \rightarrow d$



top down



these are handles that we are using
 bottom up

then always left most choice ko select krenge.
 har choice ya prod. rule ko check krna hai warna fir galat ho jayega.

sabse pehle i/p ko likhe fir hum possibilities ya choices nikalke dekhenge fir select krke reduce krenge.

→ In handle pruning, at each reduction step a particular substring matching the right side of a prod. rule is replaced by the symbol on the left hand side, we will select the correct handle at each step (which is the left most substring)

RSF = Right Sentential form

RSF	prod. rule	handle
(1) abcde	$A \rightarrow b$ $B \rightarrow dx$	b
(2) aAbcde	$A \rightarrow Abc$ $A \rightarrow bx$ $B \rightarrow dx$	Abc
(3) aAde	$B \rightarrow d$	d
(4) aABe	$S \rightarrow ABe$	aABe
(5) S	-	-

Ques: G: $E \rightarrow E+T | T$
 $T \rightarrow T * F | F$
 $F \rightarrow (E) | id$

find out all the handles for i/p: id * id

id * id (F → id, T → T * F)
 ↓
 F * id (T → F)
 ↓
 T * id (F → id) (E → T, F → id) choose this one.
 ↓
 T * F (T → T * F)
 ↓
 T (E → T)
 ↓
 E

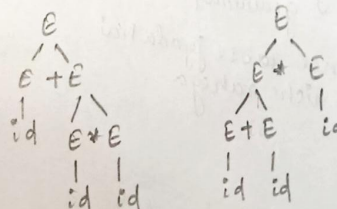
RSF	prod. rule	Handle
(i) id * id	$F \rightarrow id$	id
(ii) F * id	$T \rightarrow F$	F
(iii) T * id	$F \rightarrow id$	id
(iv) T * F	$T \rightarrow T * F$	T * F
(v) T	$E \rightarrow T$	T
(vi) E		

yahan par do choice hai but we have selected right side coz woh in the end humko E tak nhi pahunchata so sometimes we have to choose right choice also.

→ Here we have 2 choices, $E \rightarrow T$ and $F \rightarrow id$, if we replace the leftmost symbol T with E then we will not be able to reach the starting symbol E. So, here we are selecting the next symbol F → id.

Converting ambiguous grammar to unambiguous grammar:-

Ques: G: $E \rightarrow E+E | E * E | id$ i/p: id + id * id



- A grammar is ambiguous if
- (i) multiple valid LMD or RMD parse trees are possible for a single input
 - (ii) operator precedence is not clearly defined
 - (iii) operator associativity not correctly implemented

We will introduce a new non-terminal or variable that will help us in splitting the grammar into multiple levels of non-terminals based on their precedence.

Ques: $G: E \rightarrow E + E \mid E * E \mid id$
 $G': E \rightarrow E + T \mid T$
 $T \rightarrow T * F \mid F$
 $F \rightarrow id$

ipp: $id + id * id$
 dono se yahi ipp niklega
 but G' is unambiguous

We have introduced new variable T and F and kept it in different levels.
 '+' operator is closer to the starting symbol so it has lower precedence.
 '*' operator is placed farthest from the starting symbol so it has higher precedence.
 So, both '+' and '*' operators are left associativity.

Ques: $G: Bexpr \rightarrow Bexpr \text{ OR } Bexpr \mid Bexpr \text{ AND } Bexpr \mid \text{NOT } Bexpr \mid T \mid F$
 $G': Bexpr \rightarrow Bexpr \text{ OR } A \mid A$
 $A \rightarrow A \text{ AND } B \mid B$
 $B \rightarrow \text{not } B \mid T \mid F$

precedence { associativity
 (NOT > AND > OR) AND OR (L-R)
 NOT (R-L)

→ ambiguous grammar
 → jiska precedence sabse jyada hai woh sabse niche rahega

Ques: $G: E \rightarrow E \# E \mid E \& E \mid E ? E \mid E @ E \mid id$

operator precedence associativity

@ lowest right to left
 # right to left
 ? left to right
 & highest left to right

$E \rightarrow T @ E \mid T$ → new terminal T add hoga at right recursive
 $T \rightarrow F \# T \mid F$ → karna toh E nhi rahlega first mein
 $F \rightarrow F ? S \mid S$ → again new variable add krenge as (L-R) associativity hai
 $S \rightarrow S \& A \mid A$ → as (L-R) hai toh isko pehle rakhenge add krenge ek terminal naya wala
 $A \rightarrow id$ → same as above

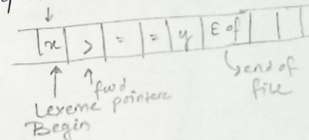
→ unambiguous grammar

Input Buffering:

A buffer contains data that is stored for a short amount of time typically in the computer's memory. The purpose of a buffer is to hold data right before it is used.

→ There are two pointers that needs to be maintained -
 (i) pointer lexeme begin
 (ii) pointer forward.

(i) pointer Lexeme begin: it marks the beginning of a current lexeme whose length we are trying to determine
 (ii) pointer forward: it scans ahead the input until a pattern is matched.



23/02/22

LA
SA
ICG
CG
TCG

Ques: Given the line of code, write down all the outputs of every phase in compiler.

$x = a + b * c$ ← HLL

Lexical Analyzer

Tokenization
(7 tokens)

$id_1 = id_2 + id_3 * id_4$

$id_1 = id_2 + id_3 * id_4$

Syntax Analyzer

← parse tree generate krega
← o/p ayega parse tree

Semantically verified
parse tree

$id_1 = id_2 + id_3 * id_4$

Intermediate Code generator

3add code

$t_0 = b * c$

$t_1 = a + t_0$

$x = t_1$

Code Optimizer

$t_0 = b * c$
 $x = a + t_0$

Target Code Generator

assembly lang.

TCG ka kaam machine code mein convert krna hota hai

MUL R_1, R_2
ADD R_0, R_1
MOV R_2, x
 $a \rightarrow R_0$
 $b \rightarrow R_1$
 $c \rightarrow R_2$

Ques: Given the arithmetic expression, write down all the o/p of all the phases in compiler.
 $a = a * -50 + b + 3$ (10 tokens)

$a = a * -50 + b + 3$

LA

$id_1 = id_1 * -50 + id_2 + 3$

SA

Syntax Analyzer

Parse tree structure for $a = a * -50 + b + 3$

Semantic Analyzer

(Semantically verified)

ICG

$t_0 = -50$
 $t_1 = a * t_0$
 $t_2 = t_1 + b$
 $t_3 = 3 + t_2$

$((a * -50) \vee$ likh skte hain)

Code optimizer

$a = a * -50 + b + 3$

TCG

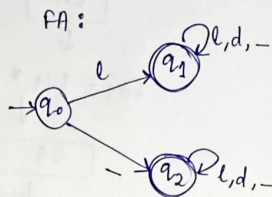
assembly lang.

STR $-50, R_2$
MUL R_1, R_0
ADD R_2, R_0
ADD $3, R_0$

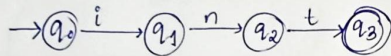
24/03/21

Regular Expression and Finite Automata

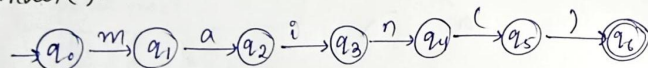
(1) Identifiers: letters/l: a|b|...|x-A|B|...|Z RE: (l|_|)(l|d|_|)*
 [a-zA-Z]
 digit/d: 0|1|2|...|9 RE: (l|_|)(l|d|_|)*
 [0-9]
 special symbol: underscore (-)
 FA:



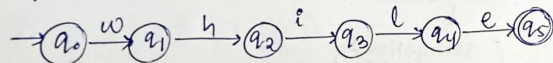
FA of int:



FA of main():



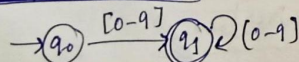
FA of while:



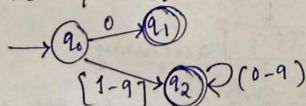
RE: (1) numbers: 0|1|2|...|9

RE: 0|[0-9]*
 RE: [0-9]⁺
 RE: 0|[1-9][0-9]*
 ye isliye nhi hoga bcoz isme E generate ho rha hai and E is not a number
 RE: [0-9]*
 ye isliye nhi hoga bcoz isme E generate ho rha hai which is not a number

FA of RE: [0-9]⁺



FA of RE: 0|[1-9][0-9]*



Imp (2) Fixed point and floating point numbers:

There is no fixed RE for this

RE: 0|[1-9][0-9]*.[0-9]⁺ → this RE is not dealing with trailing 0's.
 ex: 1.0000...

RE: 0|[0-9]*.[1-9]⁺ → 0.01 nhi hoga
 RE: 0|[0-9]*.[0-9][1-9]⁺ → 1.01

trailing 0's is not dealt in this

dot se pehle ek '1' hoga so that kuch meaning na nikal ke adhe

(3) Real Numbers:

components of RE of real nos:

- (i) integers
- (ii) floating nos.
- (iii) exponent ~~with~~ part with signs

This '1' is used to escape special meaning of '.' in the RE.

Ques: Write down the RE for signed and unsigned numbers along with construct the FA.

26/03/21

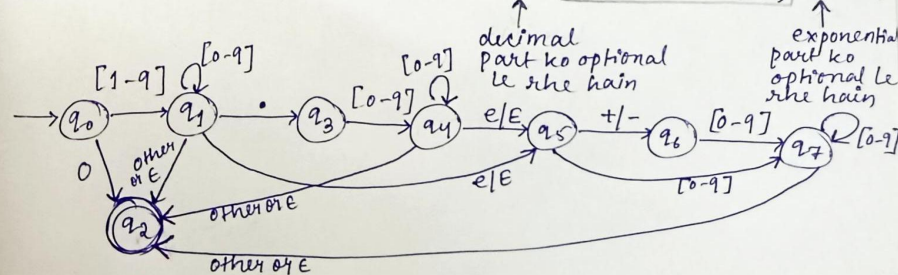
(3) Real Numbers: this '?' indicate that the [+ -] is optional.

RE: ([E][+ -]?[0-9]⁺)?

final RE:

0|[1-9][0-9]*.[0-9]⁺ ([E][+ -]?[0-9]⁺)?

↑ still humko ek real no. milega as ye whole optional hoga tab bhi humko real no milayega.

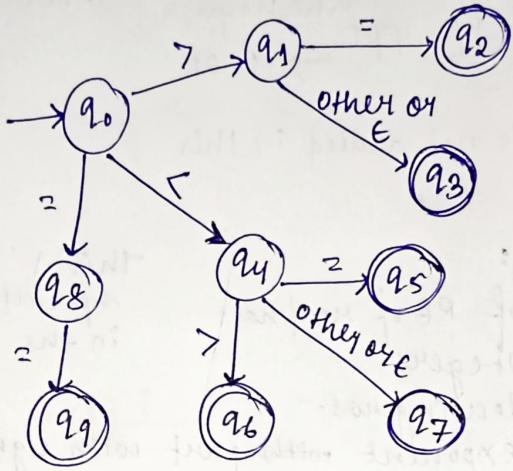


(4) Relational operator:

RE: $> | < | >= | <= | = | < \neq$

→ not

FA:

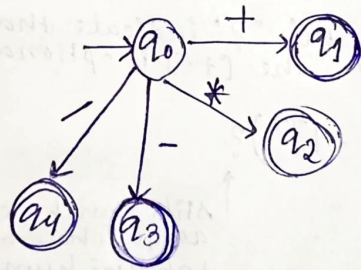


operators:

RE: $| + | * | - | /$

→ har operators se pehle '1' hakega

FA:



Ques: $G: S \rightarrow SS+ | SS* | a$